Software Testing and Validation

Second Project — Report

Dinis Rodrigues (ist106213) Artur Krystopchuk (ist104145)
Gabriel Santos (ist1116334)

5 June 2026

Abstract

This report presents the design of test cases for the urban delivery system described in the specification of the second project for the Software Testing and Validation course. Appropriate design strategies are applied to test the Order and Delivery classes at the class level, as well as the `computeDeliveryCost()` and `updateItem()` methods. The final section presents the TestNG implementation of eight selected test cases from the test suite of the Order class.

Contents

1 Class-Level Testing of the Order Class **2**
1.1 Chosen Strategy: Non-Modal Class Testing . . . 2
1.2 On/Off Points . . . 2
1.3 Domain Matrix with Test Vectors . . . 2
1.4 Relational Invariant (Same Supplier) . . . 4

2 Class-Level Testing of the Delivery Class **4**
2.1 State Machine . . . 4
2.2 Transition tree . . . 4
2.3 Testing strategy: Modal Class Testing . . . 5
2.4 Test Cases . . . 6

3 Method `computeDeliveryCost()` **7**
3.1 Chosen Strategy: Combinational Function Testing . . . 7
3.2 Decision Tree and Decision Table . . . 7
3.3 Domain Matrices . . . 9
3.4 Test Cases . . . 16

4 Method `updateItem()` **17**
4.1 Chosen Strategy: Combinational State-Input Testing & Boundary Analysis . . . 17
4.2 State + Input Combinations Matrix . . . 17
4.3 Test Cases Description . . . 18

5 Implementation in TestNG **19**

1 Class-Level Testing of the **Order** Class

1.1 Chosen Strategy: Non-Modal Class Testing

The Order class does not possess a state machine (there are no modes that alter method behaviour); its class-level behaviour is determined by a set of *invariants* over its instance variables. The appropriate strategy for testing a class of this type is **Non-Modal Class Testing**, which consists of: (i) identifying the domains of the variables; (ii) determining the *On*/*Off* points for each invariant boundary; and (iii) constructing a *domain matrix* with test vectors, where each test case exercises one boundary at a time, keeping the remaining variables at typical values.

1.2 On/Off Points

Boundary (invariant)	**On**	**Off**
client $\neq$ `null`	Valid client	`null`
addr $\neq$ `null`	Valid string	`null`
addrLength ≥ 1	1	0 (empty)
distance ≥ 0	0	-1
distance ≤ 20000	20000	20001
quantity ≥ 1	1	0
quantity ≤ 10	10	11
cost ≤ 1000	1000	1000.01
totalQty $\leq 20 + \lfloor \text{cost}/50 \rfloor$	= limit	limit + 1

1.3 Domain Matrix with Test Vectors

Each column represents a test case. In each column, exactly one boundary is in *On* or *Off* (shaded cell), while the remaining variables are at a typical value (row *In*). Adopted typical value: order with one item of price 50 and quantity 4 (cost = 200, totalQty = 4, limit = 24), address length of 3, and distance = 5000.

Table 2: Domain matrix for the Order class.

Var.	Condition		1	2	3	4	5	6	7	8	9	10
addr	≥ 1	On	1									
		Off		0								
	Typical	In			3	3	3	3	3	3	3	3
distance	≥ 0	On			0							
		Off				−1						
	≤ 20000	On					20000					
		Off						20001				
	Typical	In	5000	5000					5000	5000	5000	5000
quantity	≥ 1	On							1			
		Off								0		
	≤ 10	On									10	
		Off										11
	Typical	In	4	4	4	4	4	4				
cost	≤ 1000	On										
		Off										
	Typical	In	200	200	200	200	200	200	50	0	500	550
totalQty	$\leq$ lim.	On										
		Off										
	Typical	In	4	4	4	4	4	4	1	0	10	11
Expected result			✓	✗	✓	✗	✓	✗	✓	✗	✓	✗

Table 3: Domain matrix for the Order class (continuation).

Var.	Condition		11	12	13	14	15	16	17	18
client	$\neq$ `null`	On					ok			
		Off						`null`		
	Typical	In	ok	ok	ok	ok			ok	ok
addr	$\neq$ `null`	On							`"x"`	
		Off								`null`
	typical length	In	3	3	3	3	3	3		
distance	Typical	In	5000	5000	5000	5000	5000	5000	5000	5000
quantity	Typical	In	—	—	—	—	—	—	—	—
cost	≤ 1000	On	1000							
		Off		1000.01						
	Typical	In			200	200	200	200	200	200
totalQty	$\leq$ lim.	On			24					
		Off				25				
	Typical	In	20	20			4	4	4	4
Expected result			✓	✗	✓	✗	✓	✗	✓	✗

Legend: ✓ = valid state (operation accepted); ✗ = invalid state (must throw `InvalidOperatio`

`nException` without changing the order state, according to I6).

1.4 Relational Invariant (Same Supplier)

This invariant is non-numeric, and therefore does not have On/Off points. It is tested with two dedicated cases:

#	Scenario	Expected Result
19	Add item from supplier S to an order already containing items from supplier S	Accepted (✓)
20	Add item from supplier S' to an order already containing items from supplier S	`InvalidOperationException` (✗)

2 Class-Level Testing of the **Delivery** Class

2.1 State Machine

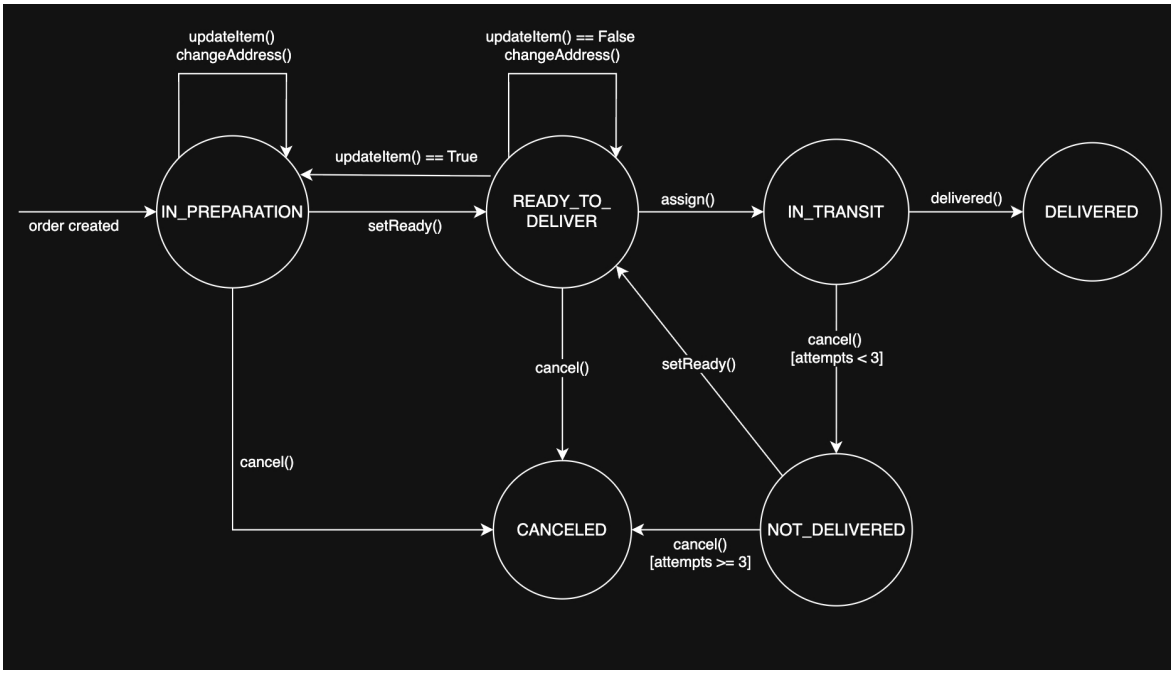


Figure 1: State machine of the Delivery class.

2.2 Transition tree

The transition tree is obtained by unfolding the state machine breadth-first from the initial state, stopping the expansion whenever a state has already appeared on the path from the root (*round-trip* criterion). Each path from the root to a leaf corresponds to one test case.

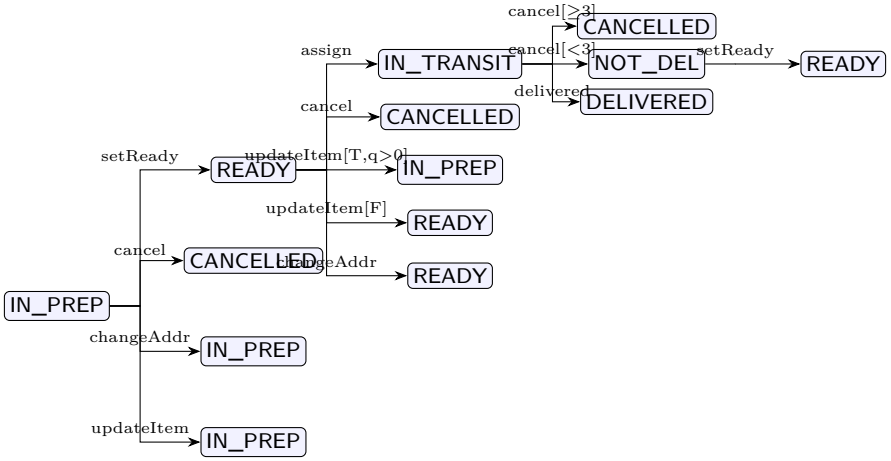


Figure 2: Transition tree of the Delivery class.

The paths from the root to the leaves yield 14 test cases for valid transitions (Table 5).

Table 5: Test paths derived from the transition tree.

TC	Event sequence	Final state
1	new	IN_PREPARATION
2	new → updateItem	IN_PREPARATION
3	new → changeAddress	IN_PREPARATION
4	new → cancel	CANCELLED
5	new → setReady	READY_TO_DELIVER
6	new → setReady → cancel	CANCELLED
7	new → setReady → changeAddress	READY_TO_DELIVER
8	new → setReady → updateItem [false]	READY_TO_DELIVER
9	new → setReady → updateItem [true, qty>0]	IN_PREPARATION
10	new → setReady → assign	IN_TRANSIT
11	new → setReady → assign → delivered	DELIVERED
12	new → setReady → assign → cancel [att<3]	NOT_DELIVERED
13	new → setReady → assign → cancel [att≥3]	CANCELLED
14	new → setReady → assign → cancel [<3] → setReady	READY_TO_DELIVER

2.3 Testing strategy: Modal Class Testing

Unlike the Order class, the Delivery class has an explicit *state machine*: its behaviour depends on the mode the delivery is in (`IN_PREPARATION`, `READY_TO_DELIVER`, `IN_TRANSIT`, `DELIVERED`, `NOT_DELIVERED`, `CANCELLED`), and each method is valid only in a subset of these modes. The class exhibits two kinds of constraint:

- **Sequence constraints** — the validity of a method invocation depends on the sequence of prior events (for example, `assign()` is only valid after `setReady()` has been called, which is captured by the current mode);
- **Domain constraints** — the validity (or outcome) of a method depends on the current content of the object (for example, `assign()` additionally requires the courier to have fewer than 5 deliveries assigned; `cancel()` from `IN_TRANSIT` branches on the number of delivery

attempts; `updateItem()` from `READY_TO_DELIVER` returns to `IN_PREPARATION` only if the resulting quantity is positive).

The coexistence of these two kinds of constraint is the defining feature of a *modal* class, and the appropriate strategy is **Modal Class Testing**.

2.4 Test Cases

Table 6: Final test suite for the Delivery class.

TC	Event sequence	Final state	Exception?
Valid transitions (from the transition tree)			
1	new	in_preparation	No
2	new → updateItem	in_preparation	No
3	new → changeAddress	in_preparation	No
4	new → cancel	cancelled	No
5	new → setReady	ready_to_deliver	No
6	new → setReady → cancel	cancelled	No
7	new → setReady → changeAddress	ready_to_deliver	No
8	new → setReady → updateItem [false]	ready_to_deliver	No
9	new → setReady → updateItem [true, qty>0]	in_preparation	No
10	new → setReady → assign	in_transit	No
11	new → setReady → assign → delivered	delivered	No
12	new → setReady → assign → cancel [att<3]	not_delivered	No
13	new → setReady → assign → cancel [att≥3]	cancelled	No
14	new → setReady → assign → cancel [<3] → setReady	ready_to_deliver	No
Invalid invocations from `IN_PREPARATION`			
15	new → assign	in_preparation	Yes
16	new → delivered	in_preparation	Yes
Invalid invocations from `READY_TO_DELIVER`			
17	new → setReady → setReady	ready_to_deliver	Yes
18	new → setReady → delivered	ready_to_deliver	Yes
Invalid invocations from `IN_TRANSIT`			
19	new → setReady → assign → setReady	in_transit	Yes
20	new → setReady → assign → assign	in_transit	Yes
21	new → setReady → assign → changeAddress	in_transit	Yes
22	new → setReady → assign → updateItem	in_transit	Yes
Invalid invocations from `NOT_DELIVERED`			
23	new → setReady → assign → cancel [<3] → assign	not_delivered	Yes

TC	Event sequence	Final state	Exception?
24	new → setReady → assign → cancel [<3] → delivered	not_delivered	Yes
25	new → setReady → assign → cancel [<3] → cancel	not_delivered	Yes
26	new → setReady → assign → cancel [<3] → changeAddress	not_delivered	Yes
27	new → setReady → assign → cancel [<3] → updateItem	not_delivered	Yes
Invalid invocations from `DELIVERED` (terminal)			
28	new → setReady → assign → delivered → setReady	delivered	Yes
29	new → setReady → assign → delivered → assign	delivered	Yes
30	new → setReady → assign → delivered → delivered	delivered	Yes
31	new → setReady → assign → delivered → cancel	delivered	Yes
32	new → setReady → assign → delivered → changeAddress	delivered	Yes
33	new → setReady → assign → delivered → updateItem	delivered	Yes
Invalid invocations from `CANCELLED` (terminal)			
34	new → cancel → setReady	cancelled	Yes
35	new → cancel → assign	cancelled	Yes
36	new → cancel → delivered	cancelled	Yes
37	new → cancel → cancel	cancelled	Yes
38	new → cancel → changeAddress	cancelled	Yes
39	new → cancel → updateItem	cancelled	Yes

3 Method `computeDeliveryCost()`

3.1 Chosen Strategy: Combinational Function Testing

The selected method-scope strategy is **Combinational Function Testing**. This is the most appropriate strategy because the method implements a business rule equivalent to a decision table: combinations of predicates over the Order state select one of several possible delivery fees. Domain Analysis is then applied inside each variant to choose On and Off boundary points systematically.

3.2 Decision Tree and Decision Table

The analysis use the concrete units from the provided interfaces: delivery distance is in metres, order cost is in cents, total quantity is counted in item units, and total weight is in grams. The expected delivery cost remains in euros, as specified for `computeDeliveryCost()`.

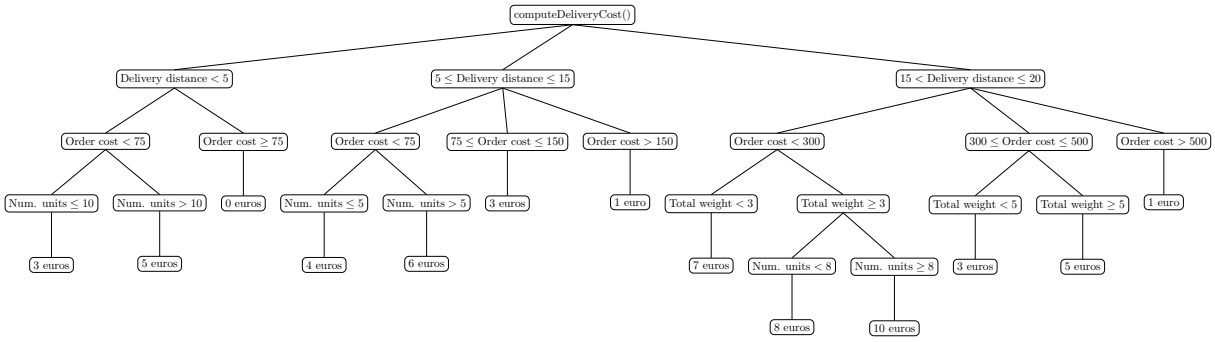


Figure 3: Decision tree for computing delivery costs based on distance, cost, and units.

Table 7: Decision table for `computeDeliveryCost()`.

Variant	**Condition**	**Expected delivery cost**
Variant 1	Delivery distance $< 5000 \wedge$ Order cost $< 7500 \wedge$ Num. units ≤ 10	3 €
Variant 2	Delivery distance $< 5000 \wedge$ Order cost $< 7500 \wedge$ Num. units > 10	5 €
Variant 3	Delivery distance $< 5000 \wedge$ Order cost ≥ 7500	0 €
Variant 4	$5000 \leq$ Delivery distance $\leq 15000 \wedge$ Order cost $< 7500 \wedge$ Num. units ≤ 5	4 €
Variant 5	$5000 \leq$ Delivery distance $\leq 15000 \wedge$ Order cost $< 7500 \wedge$ Num. units > 5	6 €
Variant 6	$5000 \leq$ Delivery distance $\leq 15000 \wedge$ $7500 \leq$ Order cost ≤ 15000	3 €
Variant 7	$5000 \leq$ Delivery distance $\leq 15000 \wedge$ Order cost > 15000	1 €
Variant 8	$15000 <$ Delivery distance $\leq 20000 \wedge$ Order cost $< 30000 \wedge$ Total weight < 3000	7 €
Variant 9	$15000 <$ Delivery distance $\leq 20000 \wedge$ Order cost $< 30000 \wedge$ Total weight $\geq 3000 \wedge$ Num. units < 8	8 €
Variant 10	$15000 <$ Delivery distance $\leq 20000 \wedge$ Order cost $< 30000 \wedge$ Total weight $\geq 3000 \wedge$ Num. units ≥ 8	10 €

Continued on next page

Table 7: Decision table for `computeDeliveryCost()`. (Continued)

Variant	Condition	Expected delivery cost
Variant 11	$15000 < \text{Delivery distance} \leq 20000 \wedge$ $30000 \leq \text{Order cost} \leq 50000 \wedge$ $\text{Total weight} < 5000$	3€
Variant 12	$15000 < \text{Delivery distance} \leq 20000 \wedge$ $30000 \leq \text{Order cost} \leq 50000 \wedge$ $\text{Total weight} \geq 5000$	5€
Variant 13	$15000 < \text{Delivery distance} \leq 20000 \wedge$ $\text{Order cost} > 50000$	1€

3.3 Domain Matrices

Table 8: Variant 1 Constraints: Delivery distance $< 5000 \wedge$ Order cost $< 7500 \wedge$ Num. units ≤ 10

V1			**-**	**1**	**-**	**2**	**3**	**-**
dist	< 5000	ON	5000					
		OFF		4999				
	IN				2001	2002	2501	2502
cost	< 7500	ON			7500			
		OFF				7499		
	IN		3001	3002			3501	3502
units	≤ 10	ON					10	
		OFF						11
	IN		2	3	4	5		
weight	IN		1001	1002	1101	1102	1201	1202
Expected results (€)			V4	3	V3	3	3	V2

Table 9: Variant 2 Constraints: Delivery distance $< 5000 \wedge$ Order cost $< 7500 \wedge$ Num. units > 10

V2			**-**	**4**	**-**	**5**	**-**	**6**
dist	< 5000	ON	5000					
		OFF		4999				
	IN				2101	2102	2601	2602
cost	< 7500	ON			7500			
		OFF				7499		
	IN		3101	3102			3601	3602
units	> 10	ON					10	
		OFF						11
	IN		12	13	14	15		
weight	IN		1301	1302	1401	1402	1501	1502
Expected results (€)			V5	5	V3	5	V1	5

Table 10: Variant 3 Constraints: Delivery distance $< 5000 \wedge$ Order cost ≥ 7500

V3			**-**	**7**	**8**	**-**
dist	< 5000	ON	5000			
		OFF		4999		
	IN				2201	2202
cost	≥ 7500	ON			7500	
		OFF				7499
	IN		10001	10002		
units	IN		2	3	4	5
weight	IN		1601	1602	1701	1702
Expected results (€)			V6	0	0	V1

Table 11: Variant 4 Constraints: $5000 \leq$ Delivery distance $\leq 15000 \wedge$ Order cost $< 7500 \wedge$ Num. units ≤ 5

V4			**9**	**-**	**10**	**-**	**-**	**11**	**12**	**-**
dist	≥ 5000	ON	5000							
		OFF		4999						
	≤ 15000	ON			15000					
		OFF				15001				
	IN						8001	8002	8501	8502
cost	< 7500	ON					7500			
		OFF						7499		
	IN		3201	3202	3301	3302			3401	3402
units	≤ 5	ON							5	
		OFF								6
	IN		1	2	3	4	1	2		
weight	IN		1001	1002	1101	1102	1201	1202	1301	1302
Expected results (€)			4	V1	4	V8	V6	4	4	V5

Table 12: Variant 5 Constraints: $5000 \leq$ Delivery distance $\leq 15000 \wedge$ Order cost $< 7500 \wedge$ Num. units > 5

V5			**13**	**-**	**14**	**-**	**-**	**15**	**-**	**16**
dist	≥ 5000	ON	5000							
		OFF		4999						
	≤ 15000	ON			15000					
		OFF				15001				
	IN						9001	9002	9501	9502
cost	< 7500	ON					7500			
		OFF						7499		
	IN		3251	3252	3351	3352			3451	3452
units	> 5	ON							5	
		OFF								6
	IN		7	8	9	10	11	12		
weight	IN		1401	1402	1501	1502	1601	1602	1701	1702
Expected results (€)			6	V1	6	V8	V6	6	V4	6

Table 13: Variant 6 Constraints: $5000 \leq$ Delivery distance $\leq 15000 \wedge$ $7500 \leq$ Order cost ≤ 15000

V6			**17**	**-**	**18**	**-**	**19**	**-**	**20**	**-**
dist	≥ 5000	ON	5000							
		OFF		4999						
	≤ 15000	ON			15000					
		OFF				15001				
	IN						10001	10002	10501	10502
cost	≥ 7500	ON					7500			
		OFF						7499		
	≤ 15000	ON							15000	
		OFF								15001
	IN		11001	11002	11501	11502				
units	IN		2	3	4	5	2	3	4	5
weight	IN		1001	1002	1101	1102	1201	1202	1301	1302
Expected results (€)			3	V3	3	V8	3	V4	3	V7

Table 14: Variant 7 Constraints: $5000 \leq$ Delivery distance $\leq 15000 \wedge$ Order cost > 15000

V7			**21**	**-**	**22**	**-**	**-**	**23**
dist	≥ 5000	ON	5000					
		OFF		4999				
	≤ 15000	ON			15000			
		OFF				15001		
	IN						12001	12002
cost	> 15000	ON					15000	
		OFF						15001
	IN		20001	20002	25001	25002		
units	IN		2	3	4	5	6	7
weight	IN		1001	1002	1101	1102	1201	1202
Expected results (€)			1	V3	1	V8	V6	1

Table 15: Variant 8 Constraints: 15000 < Delivery distance ≤ 20000 ∧ Order cost < 30000 ∧ Total weight < 3000

V8			**-**	**24**	**25**	**26**	**-**	**27**	**-**	**28**
dist	> 15000	ON	15000							
		OFF		15001						
	≤ 20000	ON			20000					
		OFF				20001				
	IN						17001	17002	17501	17502
cost	< 30000	ON					30000			
		OFF						29999		
	IN		14001	14002	14501	14502			14801	14802
weight	< 3000	ON							3000	
		OFF								2999
	IN		1001	1002	1501	1502	2001	2002		
units	IN		2	3	4	5	6	7	2	3
Expected results (€)			V6	7	7	IOE[a]	V11	7	V9	7

[a] InvalidOperationException

Table 16: Variant 9 Constraints: 15000 < Delivery distance ≤ 20000 ∧ Order cost < 30000 ∧ Total weight ≥ 3000 ∧ Num. units < 8

V9			**-**	**29**	**30**	**-**	**-**	**31**	**32**	**-**	**-**	**33**
dist	> 15000	ON	15000									
		OFF		15001								
	≤ 20000	ON			20000							
		OFF				20001						
	IN						18001	18002	18501	18502	19001	19002
cost	< 30000	ON					30000					
		OFF						29999				
	IN		14001	14002	14501	14502			14801	14802	14901	14902
weight	≥ 3000	ON							3000			
		OFF								2999		
	IN		4001	4002	4101	4102	4201	4202			4301	4302
units	< 8	ON									8	
		OFF										7
	IN		2	3	4	5	6	7	2	3		
Expected results (€)			V6	8	8	V8	V11	8	8	V8	V10	8

Table 17: Variant 10 Constraints: 15000 < Delivery distance ≤ 20000 ∧ Order cost < 30000 ∧ Total weight ≥ 3000 ∧ Num. units ≥ 8

V10			**-**	**34**	**35**	**-**	**-**	**36**	**37**	**-**	**38**	**-**
dist	> 15000	ON	15000									
		OFF		15001								
	≤ 20000	ON			20000							
		OFF				20001						
	IN						18001	18002	18501	18502	19001	19002
cost	< 30000	ON					30000					
		OFF						29999				
	IN		14001	14002	14501	14502			14801	14802	14901	14902
weight	≥ 3000	ON							3000			
		OFF								2999		
	IN		4001	4002	4101	4102	4201	4202			4301	4302
units	≥ 8	ON									8	
		OFF										7
	IN		10	11	12	13	14	15	16	17		
Expected results (€)			V6	10	10	V8	V11	10	10	V8	10	V9

Table 18: Variant 11 Constraints: 15000 < Delivery distance ≤ 20000 ∧ 30000 ≤ Order cost ≤ 50000 ∧ Total weight < 5000

V11			**-**	**39**	**40**	**-**	**41**	**-**	**42**	**-**	**-**	**43**
dist	> 15000	ON	15000									
		OFF		15001								
	≤ 20000	ON			20000							
		OFF				20001						
	IN						18001	18002	18501	18502	19001	19002
cost	≥ 30000	ON					30000					
		OFF						29999				
	≤ 50000	ON							50000			
		OFF								50001		
	IN		40001	40002	41001	41002					42001	42002
weight	< 5000	ON									5000	
		OFF										4999
	IN		2001	2002	2101	2102	2201	2202	2301	2302		
units	IN		2	3	4	5	6	7	8	9	10	11
Expected results (€)			V7	3	3	V8	3	V8	3	V13	V12	3

Table 19: Variant 12 Constraints: $15000 < \text{Delivery distance} \leq 20000 \wedge 30000 \leq \text{Order cost} \leq 50000 \wedge \text{Total weight} \geq 5000$

V12			**-**	**44**	**45**	**-**	**46**	**-**	**47**	**-**	**48**	**-**
dist	> 15000	ON	15000									
		OFF		15001								
	≤ 20000	ON			20000							
		OFF				20001						
	IN						18001	18002	18501	18502	19001	19002
cost	≥ 30000	ON					30000					
		OFF						29999				
	≤ 50000	ON							50000			
		OFF								50001		
	IN		40001	40002	41001	41002					42001	42002
weight	≥ 5000	ON									5000	
		OFF										4999
	IN		6001	6002	6101	6102	6201	6202	6301	6302		
units	IN		2	3	4	5	6	7	8	9	10	11
Expected results (€)			V7	5	5	V8	5	V9	5	V13	5	V11

Table 20: Variant 13 Constraints: $15000 < \text{Delivery distance} \leq 20000 \wedge \text{Order cost} > 50000$

V13			**-**	**49**	**50**	**-**	**-**	**51**
dist	> 15000	ON	15000					
		OFF		15001				
	≤ 20000	ON			20000			
		OFF				20001		
	IN						18001	18002
cost	> 50000	ON					50000	
		OFF						50001
	IN		60001	60002	65001	65002		
weight	IN		2001	2002	2101	2102	2201	2202
units	IN		2	3	4	5	6	7
Expected results (€)			V7	1	1	V8	V11	1

3.4 Test Cases

The final step of the Combinational Function Testing strategy is to extract the concrete test cases from the domain matrices. The table below lists the 28 distinct test cases generated from the first 8 variants.

Each test case corresponds to a numbered column in the domain matrices, defining the concrete inputs for Delivery Distance, Order Cost, Number of Units, and Total Weight, along with the expected outcome (either a specific delivery cost or an expected exception when a systemic boundary is violated).

Table 21: Concrete test cases derived from the domain matrices for `computeDeliveryCost()`.

Test Case	Delivery Distance	Order Cost	Num. Units	Total Weight	Expected Result
TC1	4999	3002	3	1002	3€
TC2	2002	7499	5	1102	3€
TC3	2501	3501	10	1201	3€
TC4	4999	3102	13	1302	5€
TC5	2102	7499	15	1402	5€
TC6	2602	3602	11	1502	5€
TC7	4999	10002	3	1602	0€
TC8	2202	7500	5	1702	0€
TC9	2302	10502	8	1802	0€
TC10	5000	3201	1	1001	4€
TC11	15000	7499	3	1101	4€
TC12	10001	3401	5	1301	4€
TC13	5000	3501	7	1401	6€
TC14	15000	7499	9	1501	6€
TC15	10002	3702	6	1702	6€
TC16	5000	7500	2	1001	3€
TC18	15000	11501	4	1101	3€
TC19	10001	7500	2	1201	3€
TC20	10501	15000	4	1301	3€
TC21	5000	20001	2	1001	1€
TC22	15000	25001	4	1101	1€
TC23	12002	15001	7	1202	1€
TC24	15001	14002	3	1002	7€
TC25	20000	14501	4	1501	7€
TC26	20001	14502	5	1502	`InvalidOperationException`

Continued on next page

Table 21: Concrete test cases derived from the domain matrices for `computeDeliveryCost()`. (Continued)

Test Case	Delivery Distance	Order Cost	Num. Units	Total Weight	Expected Result
TC27	17002	29999	7	2002	7€
TC28	17502	14802	3	2999	7€

4 Method `updateItem()`

4.1 Chosen Strategy: Combinational State-Input Testing & Boundary Analysis

The behavior of the `updateItem()` method depends simultaneously on the current execution mode of the Delivery object, the properties of the input parameters, and an internal history tracking counter. Therefore, the chosen test design strategy is a combinational synthesis of **State-Based Contextual Constraints** and **Equivalence Partitioning / Boundary Value Analysis (BVA)** applied to the input variables.

The strategy systematically checks:

1. **Contextual Constraints:** Allowed states (`IN_PREPARATION`, `READY_TO_DELIVER`) vs. forbidden states that must safely bypass execution and throw an exception.
2. **Input Constraints:** Item supplier compatibility, unit bounds ($1 \leq Q_{item} \leq 10$) , overall positive total quantity residual ($Q_{total} > 0$) , and negative input tracking for item subtraction.
3. **History Constraints:** Safe blocking execution after exactly 3 successful updates have transpired.

4.2 State + Input Combinations Matrix

Let U represent the cumulative count of successful `updateItem()` operations executed on a single instance ($0 \leq U \leq 4$). Let Q_{item} represent the requested item quantity modification, and let Q_{final_total} represent the prospective aggregated total units of all items combined in the delivery after update execution.

The logical vectors governing the input domain matrix partitions are mapped below:

Variable / Context	Partition	Values / Conditions	Expected Outcome Category
System Mode	Valid context	`IN_PREPARATION`, `READY_TO_DELIVER`	Process business rules
	Invalid context	Other FSM states	Throw `InvalidInvocationException`
Update Counter (U)	$U < 3$	$0, 1, 2$ updates achieved	Allowed to succeed if bounds valid
	$U \geq 3$	3 updates already processed	Operation fails (returns `false`)
Input Quantity (Q_{item})	Under Bound	≤ 0 (if subtraction results in total ≤ 0)	Operation fails (returns `false`)

Variable / Context	Partition	Values / Conditions	Expected Outcome Category
	Valid Positive	$1 \leq Q_{item} \leq 10$	Success; changes state if ready
	Valid Negative	$-10 \leq Q_{item} \leq -1$ (with total > 0)	Success; mode unchanged
	Over Bound	> 10	Operation fails (returns `false`)
Total Residual (Q_{final_total})	Valid	> 0 units remaining	Success
	Invalid	≤ 0 units remaining	Operation fails (returns `false`)

4.3 Test Cases Description

The following specific test vectors systematically exercise the boundary combinations derived from the strategy:

TC #	Initial Mode	Prior U	Input Q_{item}	Expected Q_{final_total}	Expected Outcome / Side Effect
4.3.1	`IN_PREPARATION`	0	5	Valid (> 0)	Returns `true`. Mode remains `IN_PREPARATION`.
4.3.2	`READY_TO_DELIVER`	0	3	Valid (> 0)	Returns `true`. Mode reverts back to `IN_PREPARATION`.
4.3.3	`READY_TO_DELIVER`	1	-2	Valid (> 0)	Returns `true`. Mode remains `READY_TO_DELIVER`.
4.3.4	`IN_PREPARATION`	0	11	Valid (> 0)	Returns `false`. Bound violated ($Q_{item} > 10$). State unchanged.
4.3.5	`IN_PREPARATION`	2	-5	Invalid (≤ 0)	Returns `false`. Total residual unit breakdown (≤ 0). State unchanged.
4.3.6	`READY_TO_DELIVER`	3	2	Valid (> 0)	Returns `false`. Saturation limit met ($U = 3$). State unchanged.
4.3.7	`IN_TRANSIT`	0	4	Valid (> 0)	Throws `InvalidInvocationException`. State unchanged.
4.3.8	`CANCELLED`	0	1	Valid (> 0)	Throws `InvalidInvocationException`. State unchanged.

5 Implementation in TestNG

Four successful cases and four unsuccessful cases were selected from the test suite of the Order class (Section 1).

```
package supereats.tests;

import org.testng.annotations.*;
import static org.testng.Assert.*;

import supereats.core.*;

public class OrderTest {

  private static final double EPS = 1e-9;
  private Supplier s1;       // reference supplier
  private Supplier s2;       // distinct supplier
  private Client   client;
  // Items from supplier s1 (weight in grams, integer price)
  private Item i1;           // price 10
  private Item i2;           // price  5
  private Item i50;          // price 50 (for totalQty cases)

  // Item from a distinct supplier (s2)
  private Item iOther;

  @BeforeMethod
  public void setUp() {
    s1     = new Supplier("Loja A");
    s2     = new Supplier("Loja B");
    client = newClient();

    i1     = new Item(100,  10, s1, "Item 1");
    i2     = new Item(200,   5, s1, "Item 2");
    i50    = new Item(150,  50, s1, "Item 50");
    iOther = new Item(100,  10, s2, "Item from another supplier");
  }

  % Helper: the existence of a Client constructor is assumed.
  private Client newClient() {
    return new Client(/* ... valid data ... */);
  }

  // ------------------------------------------------------------------------
  // SUCCESS Cases
  // ------------------------------------------------------------------------

  // TC1: valid construction - all invariants satisfied.
  @Test
  public void tc1_validConstruction() {
    Order o = new Order(client, "Street A", 5000);

    assertEquals(o.totalQuantity(), 0);
    assertEquals(o.cost(), 0.0, EPS);
    assertTrue(o.itens().isEmpty());
  }

  // TC9: addItem in an empty order with a valid quantity.
  @Test
  public void TC9AddItemEmptyOrderValidQuantity() {
    Order o = new Order(client, "Street A", 5000);

    o.addItem(i1, 1);
```

```
    assertTrue(o.contains(i1));
    assertEquals(o.quantity(i1), 1);
    assertEquals(o.cost(), 10.0, EPS);
  }

  // TC13: boundary totalQty <= 20 + floor(cost/50) (On).
  // A unit price of 1 is used to isolate the totalQty boundary without
  // violating the cost limit (I3).
  // With cost = 24 * 1 = 24, the limit is
  // 20 + floor(24/50) = 20 + 0 = 20... therefore it is necessary to raise the
  // cost enough so that the limit reaches 24:
  //   limit = 24  <=>  floor(cost/50) = 4  <=>  cost in [200, 250).
  // Cost = 240 is chosen with items of price 10 (24 units * 10 = 240),
  // keeping cost <= 1000.
  @Test
  public void TC13TotalQuantityAtDynamicLimit() {
    Order o = new Order(client, "Street A", 5000);
    // 3 items of price 10, quantities 10 + 10 + 4 = 24 units
    Item a = new Item(150, 10, s1, "A");
    Item b = new Item(150, 10, s1, "B");
    Item c = new Item(150, 10, s1, "C");

    o.addItem(a, 10);
    o.addItem(b, 10);
    o.addItem(c,  4);

    assertEquals(o.totalQuantity(), 24); // = 20 + floor (240/50) = 24
    assertEquals(o.cost(), 240.0, EPS);
  }

  // TC19: adding an item from the SAME supplier is accepted.
  @Test
  public void TC19SameSupplierIsAccepted() {
    Order o = new Order(client, "Street A", 5000);

    o.addItem(i1, 2);
    o.addItem(i2, 3); // i2 also from supplier s1

    assertTrue(o.contains(i1));
    assertTrue(o.contains(i2));
    assertEquals(o.totalQuantity(), 5);
    assertEquals(o.cost(), 2 * 10 + 3 * 5, EPS);
  }

  // ------------------------------------------------------------------------
  // FAILURE Cases
  // ------------------------------------------------------------------------

  // TC2: boundary addrLength >= 1 (Off) - empty address.
  @Test(expectedExceptions = InvalidOperationException.class)
  public void TC2EmptyAddressThrows() {
    new Order(client, "", 5000);
  }

  // TC6: null client.
  @Test(expectedExceptions = InvalidOperationException.class)
  public void TC6NullClientThrows() {
    new Order(null, "Street A", 5000);
  }

  // TC10: boundary quantity <= 10 (Off) - quantity 11.
  @Test
  public void TC10QuantityAboveMaxThrowsAndKeepsState() {
```

```
122     Order o = new Order(client, "Street A", 5000);
123     try {
124       o.addItem(i1, 11);
125       fail("Expected InvalidOperationException");
126     } catch (InvalidOperationException e) {
127       assertFalse(o.contains(i1)); // state unchanged
128       assertEquals(o.totalQuantity(), 0);
129     }
130   }
131
132   // TC20: adding an item from a DISTINCT supplier.
133   @Test(expectedExceptions = InvalidOperationException.class)
134   public void TC20DifferentSupplierThrows() {
135     Order o = new Order(client, "Street A", 5000);
136     o.addItem(i1,     1); // supplier s1
137     o.addItem(iOther, 1); // supplier s2 -> exception
138   }
139 }
```

Listing 1: Skeleton of the TestNG test class for Order.